

# DIG-TAG: REPRESENTING OPEN-ENDED COLLABORATION IN LLM-BASED MULTI-AGENT SYSTEMS STRUCTURALLY

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

LLM multi-agent systems collaborate through open-ended interaction, creating and transforming tasks on the fly as they act in shared environments. Existing approaches typically model environment state while leaving the collaborative state spanning agents and their shared environment implicit. We introduce DIG-TAG, a representation that makes emergent collaboration explicit while leaving agent reasoning and behavior open-ended. It couples a Dynamic Interaction Graph (DIG) with a Task Activity Graph (TAG), connecting agent interaction and agent-generated task evolution with evidence and the shared environment. We further develop a method that directly optimizes over the collaborative state made explicit by DIG-TAG. Experiments across diverse environments show that DIG-TAG-based optimization improves grounded task completion over baselines. DIG-TAG provides a formal and empirical framework for representing, studying, and optimizing collaboration.

## 1 INTRODUCTION

Large language models (LLMs) increasingly act as agents that communicate, divide work, use tools, and act together in shared environments. Their central promise is *open-ended collaboration*: agents need not receive a complete division of labor or fixed workflow in advance. During execution, they can propose tasks, revise objectives, introduce decompositions, exchange intermediate results, combine partial solutions, and reorganize after failure. Yet this flexibility creates a fundamental challenge: if collaboration is itself formed and transformed through interaction, *what computational object represents it?*

Existing representations expose only fragments of this process. Environment state describes the world, but not necessarily the collective work agents have formed; it may remain unchanged after agents revise their shared task although the appropriate next decision changes. Even when environment and task state agree, the responsible agent may or may not have received critical evidence, making the proper intervention either to proceed or to route information. A transcript preserves these distinctions but entangles them with surface language and incidental execution order; a predefined workflow makes structure explicit but fixes it before collaboration begins. The challenge is to expose collaboration formed during execution for analysis *without predefining how it must unfold*.

Our central move is to treat the externally consequential structure created by interaction as a first-class representational object, captured through two relations over one realized execution. In **DIG-TAG**, a Dynamic Interaction Graph (DIG) records the interaction-dependency flow of communication, observations, tool calls, and actions. A Task Activity Graph (TAG) selects task-typed DIG events and occurrence-identified task calls, organizing them by persistent task identity and transformation. TAG is induced from DIG rather than maintained and aligned separately, but it is not merely a causal subgraph: its task-flow relation is distinct from DIG’s interaction-flow relation and exposes structure needed for composition and optimization. The shared objects retain the provenance of every externalized task event and action. Together, the two views make collective work explicit, comparable, and optimizable without modeling private reasoning or prescribing a joint program.

A task may enter as an externally supplied initial event, but subsequent task evolution occurs through explicit task-effect calls rather than silent changes to TAG. The environment returns observations

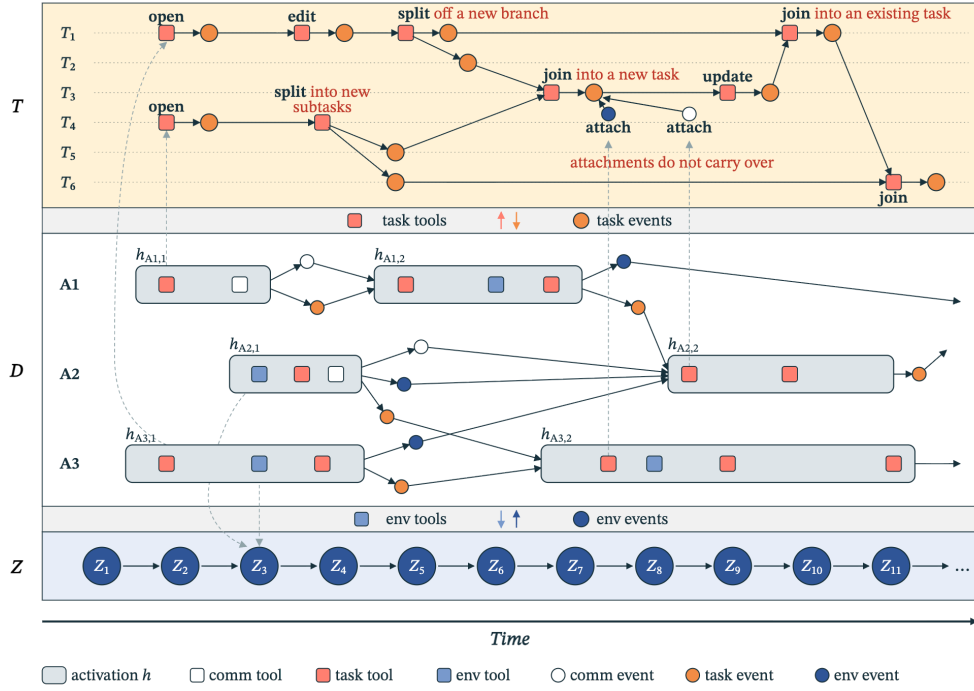


Figure 1: Enter Caption

and evidence; agents use them when declaring or revising task organization. A separate relation attaches evidence to the particular task transformation it supports and records agent or environment provenance. Provenance does not imply validity: agent evidence may remain provisional, while an environment-facing claim is grounded only when a domain verifier accepts corresponding environment evidence. The representation thereby separates what work is declared, how it arose, and what grounds it.

DIG-TAG does not presume cooperation: unsuccessful, irrelevant, or conflicting behavior remains representable, while a downstream criterion may evaluate cooperation from exposed task requirements, interaction, and grounded progress. Agents also need not reason in graphs or expose private reasoning. They may invoke task tools directly, or a wrapper or evaluated parser may emit equivalent typed events and calls from observable behavior. These labels record consequences after agents decide what to do; they neither supply a workflow nor recover effects that were never externalized.

Our contributions are:

- We formulate the representation problem for open-ended LLM-agent collaboration and introduce DIG-TAG, which factors externalized execution into complementary causal-interaction and task-activity relations without prescribing reasoning, assuming cooperation, or fixing a workflow.

## 2 RELATED WORK

**LLM multi-agent collaboration.** LangGraph supports sequential and parallel execution, conditional routing, iteration, and workers instantiated during execution (?). Related mechanisms appear in CrewAI, Microsoft Agent Framework, the OpenAI Agents SDK, and Google ADK (????). These mechanisms determine which agents act and how information is routed. We study whether their realized collaboration structures admit a common DIG-TAG representation, including when routing, task allocation, and the number of invocations depend on intermediate results.

**Planning and agent-task representations.** PDDL; multi-agent planning; partial-order planning (HTNs); BDI; expressiveness, compositionality, solvability, and complexity.

**Models and collaboration structure.** Petri nets; AND/OR graphs; concurrent and distributed processes; process algebra; causal structure and equivalence under irrelevant interleavings; compositional reasoning; DIG; iCORE. models: Dec-POMDP, POSG, BDI system, or extensive-form game? provenance models (PROV-style lineage, event structures, colored Petri nets, git-like version DAGs) to related work for TAG

### 3 THE DIG-TAG COLLABORATION INTERFACE

We consider asynchronously activated agents that may receive information events at different times, perform private computation, and produce or send events. DIG-TAG unifies agent interactions with one another, the environment, and task states:

$$\begin{aligned} \mathcal{DT} &= (\mathcal{D}, \mathcal{T}, \mathcal{Z}, \mathcal{U}), \quad \mathcal{D} = (\mathcal{E}, \mathcal{H}, \mathcal{L}_D), \\ \mathcal{T} &= (\mathcal{Q}, \mathcal{A}, \Phi, \mathcal{L}_T), \quad \mathcal{U} = \mathcal{U}_T \cup \mathcal{U}_Z \cup \mathcal{U}_\phi. \end{aligned} \quad (1)$$

Here  $\mathcal{D}$ ,  $\mathcal{T}$ , and  $\mathcal{Z}$  are the Dynamic Interaction Graph, Task Activity Graph, and shared environment, respectively, and  $\mathcal{U}$  is the tool-action space. The components share persistent agent identities  $\mathcal{I}$ . Every tool is called by an agent in  $\mathcal{D}$ , returns information that  $\mathcal{D}$  records as events, and may additionally update  $\mathcal{T}$  or  $\mathcal{Z}$ . Full typing and transition semantics appear in Appendix A.

**DIG: Dynamic Interaction Graph.** The DIG  $\mathcal{D}$  is a directed bipartite graph of information events and agent activations. Its edges record how events flow between activations. An activation  $h_{[t:t']} \in \mathcal{H}$  records one invocation of agent  $i \in \mathcal{I}$ :

$$h_{[t:t']} = (i, E_h, U_h), \quad U_h = \langle (u_r, E_r) \rangle_{r=1}^m, \quad u_r \in \mathcal{U}. \quad (2)$$

Here  $E_h \subseteq \mathcal{E}$  is the set of input events for  $h$ , and  $U_h$  is the ordered sequence of tool calls paired with their returned event sets  $E_r \subseteq \mathcal{E}$ . Events may carry arbitrary payloads.

**TAG: Task Activity Graph.** The TAG  $\mathcal{T}$  consists of a directed bipartite task graph and its evidence attachments. Its objects are

$$q = (k, \delta, \sigma) \in \mathcal{Q}, \quad a = (h, u) \in \mathcal{A}, \quad u \in \mathcal{U}_T, \quad \delta = (g, c), \quad c(g, \sigma) \in \Omega, \quad \phi = (s, x, q) \in \Phi, \quad (3)$$

Here  $k$  is the persistent task identity,  $g$  is the task goal,  $\sigma$  is its state, and  $c$  is an evaluation rule. OPEN creates a new task identity with status open; CLOSE( $k$ ) sets its status to closed. Each  $q$  is a version of task  $k$ ; UPDATE and EDIT append a new version with the input version as its predecessor. The result  $c(g, \sigma) \in \Omega$  may be a score or statement. Edges entering task action  $a$  identify its input versions, while edges leaving  $a$  identify the versions it creates.

Each evidence item  $\phi$  contains a source  $s$ , arbitrary payload  $x$ , and existing task node  $q$ . Attached evidence does not automatically carry over when  $q$  is updated.

**DIG-TAG: Coupled Transitions.** The DIG  $\mathcal{D}$  is the common interaction layer coupling the TAG  $\mathcal{T}$  and environment  $\mathcal{Z}$ . The information and evidence tools are

$$\mathcal{U}_\phi = \{\text{send}_h, \text{attach}_h, \text{observe}_h\}, \quad \text{send}_h(e, J), \quad \text{attach}_h(\phi), \quad \text{observe}_h, \quad e \in \mathcal{E}, \quad J \subseteq \mathcal{I}. \quad (4)$$

Here  $\text{send}_h(e; J)$  sends event  $e$  from activation  $h$  to agents in  $J$ ,  $\text{attach}_h(\phi)$  attaches evidence  $\phi$  to its target task node  $q$ , and  $\text{observe}_h$  reads  $\mathcal{T}$  without changing it.

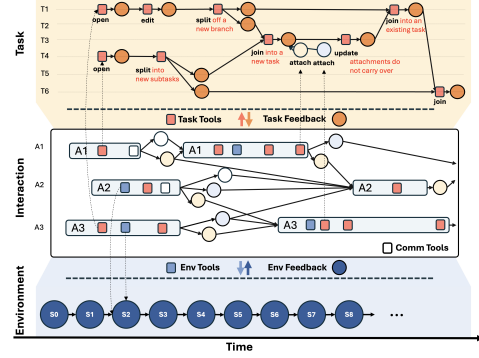


Figure 2: Tool calls may return DIG events; evidence attaches sourced payloads to task nodes.

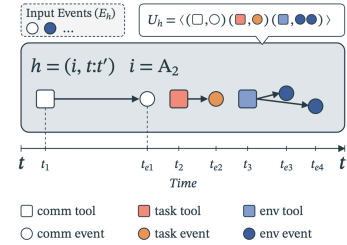


Figure 3: DIG records agent activations, input events, tool calls and their returned events.

Table 1: Task tools in  $\mathcal{U}_T$ .

| Task tool                                           | TAG effect                           |
|-----------------------------------------------------|--------------------------------------|
| OPEN( $\delta, \sigma$ )                            | $\emptyset \rightarrow q$            |
| EDIT( $\delta(q, \delta')$ )                        | $q \rightarrow q'$                   |
| UPDATE( $q, \sigma'$ )                              | $q \rightarrow q'$                   |
| SPLIT( $q, \{(k_j, \delta_j, \sigma_j)\}_{j=1}^m$ ) | $q \rightarrow \{q_1, \dots, q_m\}$  |
| JOIN( $\{q_j\}_{j=1}^m, k', \delta', \sigma'$ )     | $\{q_1, \dots, q_m\} \rightarrow q'$ |
| CLOSE( $k$ )                                        | mark $k$ closed                      |

For a tool call  $u$  issued in activation  $h$ ,  $\mathcal{D} \oplus_h (u, E_u)$  denotes its DIG update, where  $E_u \subseteq \mathcal{E}$  is the possibly empty event set returned by  $u$ . A call returns information from the domain it addresses, in that domain's form;  $\mathcal{D}$  records the returned information as fresh events, each with an identity, a payload, and a declared origin. Once in  $\mathcal{D}$ , information passes as DIG events: an event is the DIG's wrapper for information, whatever its domain. A task call  $u \in \mathcal{U}_T$  causes the TAG update  $\mathcal{T} \oplus (a, Q)$ , where  $a = (h, u)$  is its action node and  $Q$  is the set of created task nodes;  $Q$  belongs to  $\mathcal{T}$  and is separate from  $E_u$ . For an environment call  $u \in \mathcal{U}_Z$ ,  $\mathcal{Z}'$  denotes the resulting environment state. The update  $\mathcal{T} \oplus \phi$  adds evidence attachment  $\phi$ . In the send case,  $h'_j$  is the receiving activation of agent  $j \in J$ . Suppressing the unchanged tool space  $\mathcal{U}$ , the coupled update is

$$\text{DT} \xrightarrow[E_u]{h,u} \text{DT}' = \begin{cases} ((\mathcal{D} \oplus_h (u, \emptyset)) \oplus_{\{h'_j: j \in J\}} (\emptyset, \{e\}), \mathcal{T}, \mathcal{Z}), & u = \text{send}_h(e, J), \\ (\mathcal{D} \oplus_h (u, E_u), \mathcal{T} \oplus (a, Q), \mathcal{Z}), & u \in \mathcal{U}_T, \\ (\mathcal{D} \oplus_h (u, E_u), \mathcal{T}, \mathcal{Z}'), & u \in \mathcal{U}_Z, \\ (\mathcal{D} \oplus_h (u, E_u), \mathcal{T} \oplus \phi, \mathcal{Z}), & u = \text{attach}_h(\phi), \\ (\mathcal{D} \oplus_h (u, E_u), \mathcal{T}, \mathcal{Z}), & u = \text{observe}_h. \end{cases} \quad (5)$$

If  $u \in \mathcal{U}_T \cup \{\text{attach}_h, \text{observe}_h\}$ ,  $E_u$  contains information from  $\mathcal{T}$ ; if  $u \in \mathcal{U}_Z$ , it contains information from  $\mathcal{Z}$ . A SEND returns nothing, since  $e$  is already in  $\mathcal{D}$ .

## 4 CAPABILITIES OF DIG-TAG

By making collaboration explicit, DIG-TAG provides a computational basis for analyzing how agents organize their work and reasoning about how that work should proceed. This section establishes the capabilities of the representation and examines how its records can support the reconstruction, assessment, and adaptation of collaboration.

### 4.1 REPRESENTING EVOLVING COLLABORATION WITH DIG

**Collaboration as graph execution.** We view collaboration as the execution of a graph whose nodes represent agents, tools, or other computations, and whose edges represent communication, invocation, routing, or dependency relations. This graph may be prescribed by a workflow, implicit in agents' decisions, or revised during execution. The collaboration process consists of invoking these nodes and realizing their interactions.

Sequential, conditional, cyclic, and parallel patterns can coexist, as in LangGraph. Table 2 summarizes how DIG records these patterns. The representation accepts domain-specific payloads and tool implementations and any finite team; record size depends on recorded occurrences, relations, and payloads.

Table 2: Collaboration patterns and their DIG records.

| Pattern                | Recorded execution in DIG                                                                                  |
|------------------------|------------------------------------------------------------------------------------------------------------|
| Sequential             | Call order and recorded dependencies establish precedence.                                                 |
| Conditional branching  | Routing decisions identify the successors selected at each invocation.                                     |
| Cycles and retries     | Repeated invocations produce distinct activations.                                                         |
| Parallel branches      | Dependencies record precedence; common-clock intervals show overlap, and step labels identify super-steps. |
| Adaptive collaboration | New agents and routing decisions are recorded as they occur.                                               |

**Graph execution  $\rightarrow$  DIG trace.** DIG records individual occurrences, keeping repeated invocations distinct. For a finite execution  $\tau$ , let  $W_\tau$  contain invocation and call occurrences, events, and their recorded relations.

**Proposition 1.** *Any finite graph execution  $\tau$ , with its occurrences and relations recorded, admits an online DIG encoding  $\text{Enc}$  satisfying  $\Pi(\text{Enc}(\tau)) \cong W_\tau$ . The projection  $\Pi$  preserves occurrence kinds, labels, interaction relations, and available timing and step annotations.*

**DIG trace  $\rightarrow$  graph.** A DIG trace can be sliced by time and agent identity. For a time window  $[t, t']$  and a selected set of agents  $I$ , let  $S \subseteq \mathcal{H}$  contain activations of agents in  $I$  that overlap the window. Let  $i_h$  be the agent of  $h$  and  $J_h$  contain the recipients of its SEND calls issued within the window.

Merging activations by agent identity and retaining interactions within  $I$  gives

$$G_{[t:t'], I} = \left( \{i_h : h \in S\} \cup \bigcup_{h \in S} (J_h \cap I), \{(i_h, j) : h \in S, j \in J_h \cap I\} \right). \quad (6)$$

To retain tool and other computation nodes, group invocation and call occurrences by their recorded source-node labels and aggregate their relations, yielding  $G_\tau$ . Among the first  $n$  completed source invocations, let  $N_v(n)$  count invocations of  $v$  and  $C_{vw}(n)$  those selecting  $w$ .

**Proposition 2.** *For a fixed finite collaboration graph  $\Gamma$ , the graph recovered from its DIG trace satisfies  $G_\tau \subseteq \Gamma$ , with equality under complete vertex and edge coverage. If each source is invoked infinitely often and its routing choices are independent across visits with fixed inclusion probabilities  $p_{vw}$ , then, almost surely,  $\lim_{n \rightarrow \infty} C_{vw}(n)/N_v(n) = p_{vw}$  and every positive-probability route is eventually observed.*

Comparing time windows reveals changes in realized collaboration, while cumulative aggregation retains historical routes. Recording the initial configuration and ordered revisions additionally recovers the configured graph over time, including branching and synchronization rules when their definitions are retained.

## 4.2 REPRESENTING TASK EVOLUTION WITH TAG

**Collaboration as task evolution.** Collaboration involves agents pursuing shared goals through coordinated work. Task specifications, reported states, and the organization of subtasks evolve as work proceeds. TAG preserves this history through explicit transformations between task versions. Evidence linked to exact versions supports assessment of progress toward the corresponding goals.

**A general task interface.** TAG represents the structure of task updates through a fixed vocabulary. Applications supply the meaning of task specifications, reported states, and transformation results. Git provides a familiar example of a shared vocabulary for versioned collaboration; Table 3 gives corresponding interpretations for task evolution.

Table 3: Task-level interpretations of Git vocabulary.

| Git vocabulary | Role in versioned work                       | Task-level interpretation in TAG                                                                                    |
|----------------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| commit         | Record a new snapshot.                       | Create a version with EDIT, UPDATE, or their composition; OPEN introduces an initial task version.                  |
| branch         | Name a separate line of development.         | Reuse historical versions for alternative continuations; use SPLIT when the work receives distinct task identities. |
| merge          | Integrate divergent development.             | Use JOIN to integrate versions of distinct task identities.                                                         |
| revert         | Record changes that reverse earlier changes. | Create a compensating version through edits or updates while retaining the earlier history.                         |

**Proposition 3** (Structural sufficiency). *Any finite history of task introduction, specification or reported state revision, decomposition, integration, and closure that obeys TAG’s identity rules can be represented by composing TAG actions. The representation preserves task identities, original version contents and ancestry, and identity status at operation boundaries.*

Additional domain-specific verbs whose task effects lie in this class do not enlarge the set of structurally representable task histories. They supply domain meanings, constraints, and implementations for updates expressible through the same primitives.

**Proposition 4** (Version and frontier).

## 4.3 EVIDENCE $\Phi$ IN DIG-TAG

**Queries across both views.** Linking task actions to DIG call occurrences connects version lineage with interaction history.

**Proposition 5** (Structural observability).

#### 4.4 DECISION MAKING WITH DIG-TAG IN Z

**From query answers to tool calls.**

### 5 OPPORTUNITIES ENABLED BY DIG-TAG

The relations above provide inputs for methods that adapt collaboration during execution.

**Adaptive coordination.** Aggregating activations for a task, phase, or agent group gives a graph of its observed interactions. Aligned with task lineage and assessed outcomes, these graphs support comparisons of team structures and routing patterns. A controller could use such comparisons to select recipients, assign reviewers, or revise task decomposition.

**Targeted verification and repair.** A failed check or invalidated observation identifies its attached task versions for reassessment. Task lineage and recorded information flow provide candidate downstream effects, while call attribution identifies the associated agents. A repair policy could select what to recheck or revise, with domain rules determining which other results remain valid.

**Learning collaboration policies.** Records link call choices to available public information, returned observations, and declared task effects. Given domain assessments and call costs, these records could train policies for choosing actions, recipients, or further checks. Evaluation could test transfer across team structures and domains and measure the effect of removing version, provenance, or delivery information.

## 6 EXPERIMENTS

### 6.1 REPRESENTING AND RECOVERING COLLABORATION STRUCTURE

figs-topology/recovery.pdf

Figure 4: From interaction trace to topology. (a) Two rounds of hub-worker interaction produce seven activations across three agents and eight routed deliveries. (b) Aggregation by agent identity yields three nodes and four directed edges, each with delivery count 2. Matching colors connect deliveries in the trace to their aggregate edges; repeated interactions become weights, while activation timing is omitted.

### 6.1.1 TOPOLOGY AND COLLABORATION PATTERNS

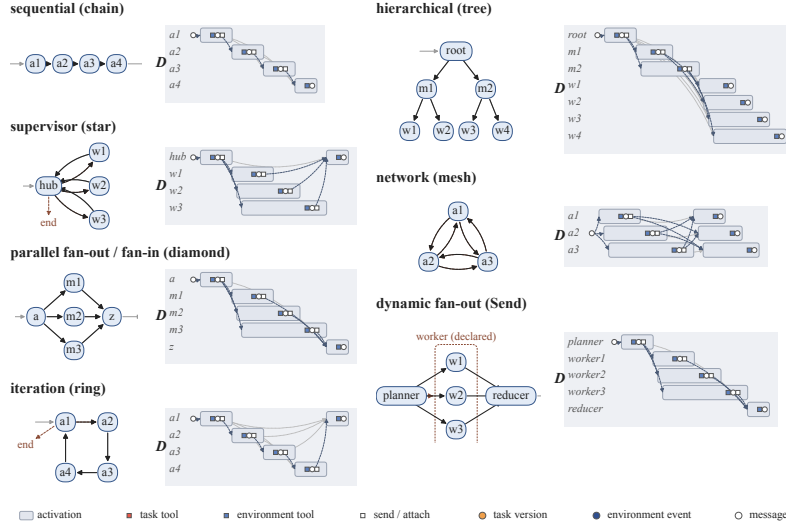


Figure 5: Seven structures and their records. Left: declared LangGraph topology with recovered interactions overlaid in black; brown dashed marks termination routes and the declared dynamic worker node. Right:  $\mathcal{D}$ , with activations in agent lanes, calls as squares (blue PUBLISH, white SEND), and returned events as circles. Dashed links show routed deliveries; dotted links show shared-state or retained context. Same-superstep activations overlap; dynamic worker lanes denote instantiated identities.

### 6.1.2 TOPOLOGY RECOVERY AND PROBABILITY ESTIMATION

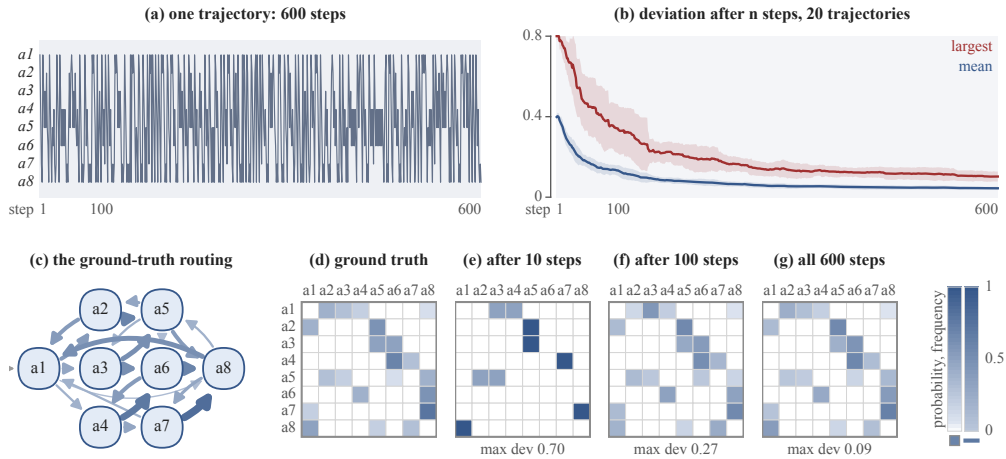


Figure 6: Recovery of routing frequencies. (a) One 600-step trajectory over eight agents. (b) Maximum and mean absolute deviations from the defined transition probabilities after  $n$  steps; curves show means and shaded bands one standard deviation across 20 trajectories. (c) The defined routing graph with 21 directed edges. (d) Defined transition probabilities. (e-g) Cumulative empirical frequencies after 10, 100, and 600 steps, normalized by source activations; rows are sources and columns targets. Below each empirical matrix is its maximum absolute deviation.

### 6.1.3 TOPOLOGY RECOVERY FROM KNOWN PHASES IN LONG HORIZON

### 6.1.4 SCALABILITY

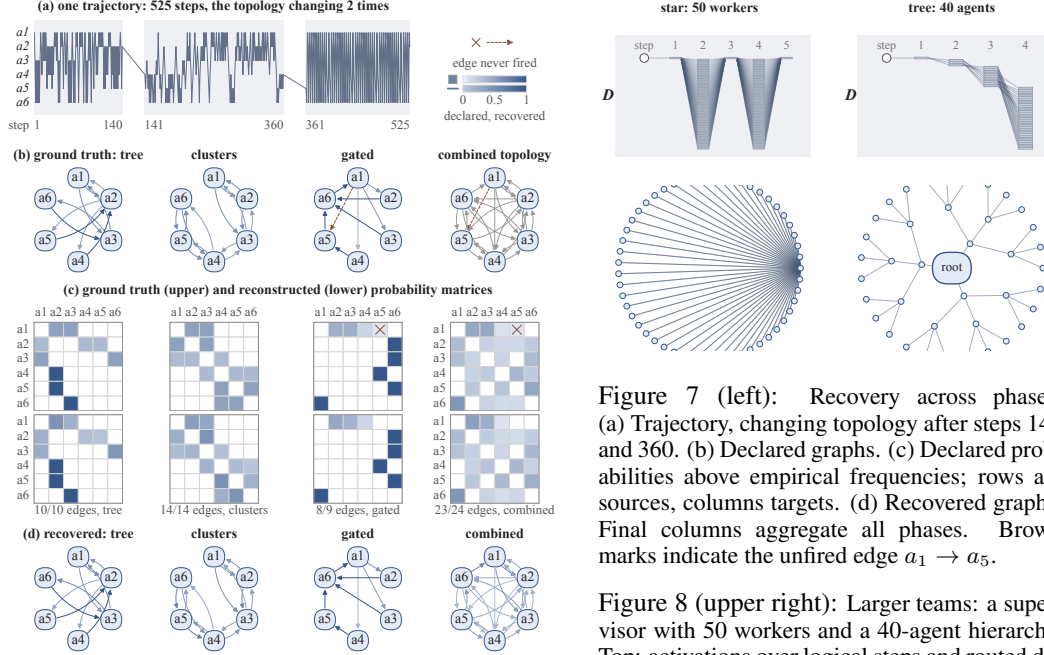


Figure 7 (left): Recovery across phases. (a) Trajectory, changing topology after steps 140 and 360. (b) Declared graphs. (c) Declared probabilities above empirical frequencies; rows are sources, columns targets. (d) Recovered graphs. Final columns aggregate all phases. Brown marks indicate the unfired edge  $a_1 \rightarrow a_5$ .

Figure 8 (upper right): Larger teams: a supervisor with 50 workers and a 40-agent hierarchy. Top: activations over logical steps and routed deliveries. Bottom: interaction graphs aggregated from the records.

## 6.2 TRACING TASK EVOLUTION

| Query                            | Answer from the record                                                                                                                                                                                |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Open identities and frontier     | After the join, $\mathcal{K} = \{k_1, \dots, k_4\}$ and $\mathcal{F} = \{q_8\}$ ; after $\text{CLOSE}(k_4)$ , $\mathcal{K} = \{k_1, k_2, k_3\}$ and $\mathcal{F} = \{q_8\}$ still.                    |
| Who produced $q_8$               | The join $a_6$ in $h_6$ (planner) from $q_5$ and $q_7$ ; $q_5$ by the update in $h_3$ (verifier) from $q_4$ , and $q_7$ by the update in $h_5$ (verifier) from $q_6$ .                                |
| Evidence and context at the join | $q_5$ carries $\varphi_2$ (source $e_2$ , the check's feedback), $q_4$ carries $\varphi_1$ (source $h_2$ , the search hits), $q_8$ none; before the join $h_6$ had $q_1, q_2, q_3, q_5$ , and $q_7$ . |

Table 4: Three queries answered

from the record of Figure 9.

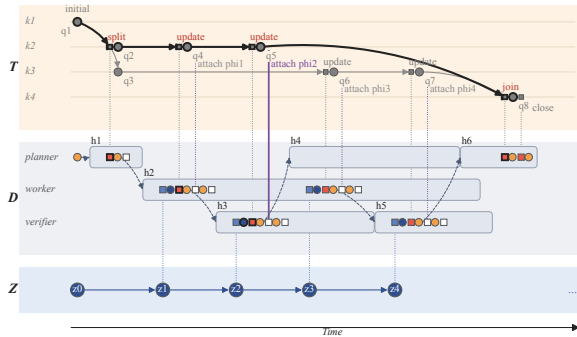


Figure 9: Task evolution ( $\mathcal{T}$ ), agent activity ( $\mathcal{D}$ ), and environment transitions ( $\mathcal{Z}$ ) in one record. Task versions occupy identity lanes; arrows show task actions, dotted links connect calls to versions, evidence, or transitions, and a cross marks closure. Task calls are red. Bold highlights the lineage  $q_1 \rightarrow q_2 \rightarrow q_4 \rightarrow q_5 \rightarrow q_8$  and verdict  $\varphi_2$  from feedback  $e_2$ .

### 6.3 DOMAIN AGNOSTICISM

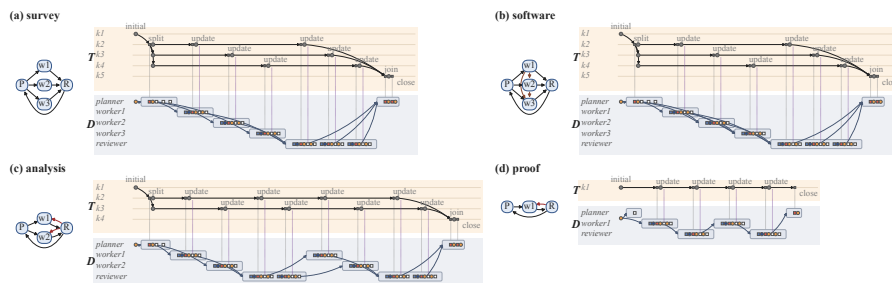
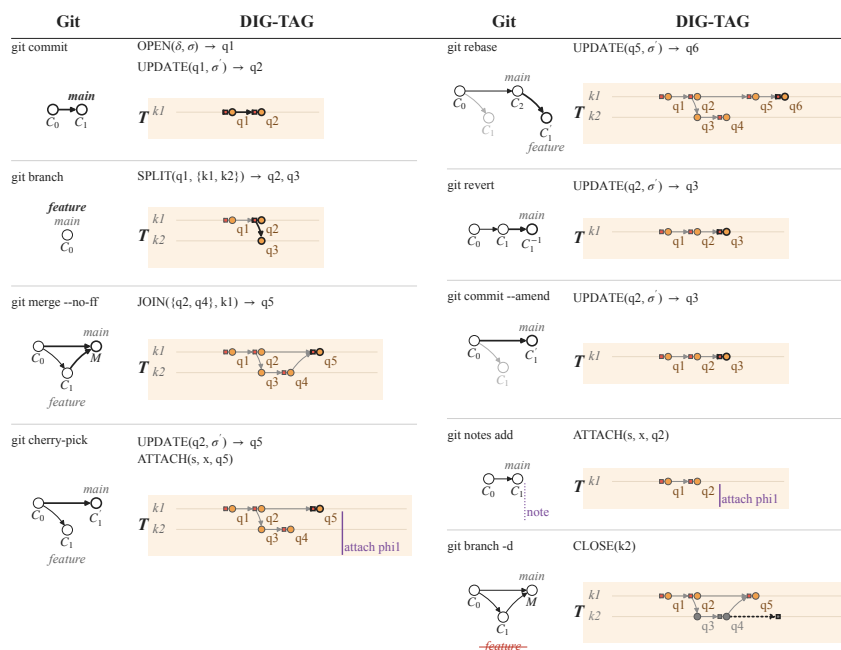


Figure 10: Shared policies across (a) survey, (b) software, (c) analysis, and (d) proof. Left: recovered interaction graphs (P planner, R reviewer; brown worker hand-offs; red rejected drafts). Right: aligned task and activity records using Figure 9’s notation.



## REFERENCES

## A DIG, TAG, AND DIG-TAG

A DIG-TAG instance is

$$DT = (\mathcal{D}, \mathcal{T}, \mathcal{Z}, \mathcal{U}), \quad \mathcal{D} = (\mathcal{E}, \mathcal{H}, \mathcal{L}_D), \quad \mathcal{T} = (\mathcal{Q}, \mathcal{A}, \Phi, \mathcal{L}_T), \quad \mathcal{U} = \mathcal{U}_T \cup \mathcal{U}_Z \cup \mathcal{U}_\phi.$$

The components share a set  $\mathcal{I}$  of persistent agent identities. The tool classes are disjoint. For a finite execution prefix,  $DT_t$  denotes the components realized by time  $t$ ; the fixed action space  $\mathcal{U}$  is suppressed when unchanged.

**Dynamic Interaction Graph (DIG).** The DIG  $\mathcal{D}$  is a directed bipartite graph of information-event nodes  $\mathcal{E}$ , agent-activation nodes  $\mathcal{H}$ , and event-flow edges  $\mathcal{L}_D$ . An activation of agent  $i$  over  $[t, t']$  is

$$h_{[t:t']} = (i, E_h, U_h) \in \mathcal{H}, \quad U_h = \langle (u_r, E_{u_r}) \rangle_{r=1}^m, \\ i \in \mathcal{I}, \quad u_r \in \mathcal{U}, \quad E_h, E_{u_r} \subseteq \mathcal{E}.$$

The input set  $E_h$  is fixed at activation start and contains only events already produced or supplied initially; additional incoming information requires a new activation. Activations may overlap. The sequence  $U_h$  orders completed call-return occurrences. For any call  $u$ , its returned event set  $E_u$  may be empty. The pair  $(h, r)$  identifies a call occurrence even when its tool and arguments repeat;  $u$  denotes the occurrence when its position is implicit.

Each event has a distinct identity, an arbitrary fixed payload, and a fixed declared origin. Every noninitial event is produced by exactly one call; distinct events may carry identical payloads. The declared origin may differ from the agent issuing the producing call and does not certify payload validity. The event-flow edges are

$$\mathcal{L}_D = \bigcup_{h \in \mathcal{H}} (\{e \rightarrow h : e \in E_h\} \cup \{h \rightarrow e : (u, E_u) \in U_h, e \in E_u\}).$$

Input edges record presentation to an activation, without asserting actual use; output edges record returns from its calls. A DIG event referenced by  $u_r$  must be an input to  $h$  or a return from an earlier call:

$$e \in E_h \cup \bigcup_{\ell < r} E_{u_\ell}.$$

The update  $\mathcal{D} \oplus_h (u, E_u)$  appends  $(u, E_u)$  to  $U_h$  and adds the fresh returned events with edges  $h \rightarrow e$  for  $e \in E_u$ . Creating an activation adds its node and input incidences.

**Tool observations.** Task calls,  $\text{attach}_h$ , and  $\text{observe}_h$  return information from  $\mathcal{T}$ , and environment calls return information from  $\mathcal{Z}$ , each obtained from an implementation-defined observation function;  $\mathcal{D}$  records the returned information as the event set  $E_u$ . These functions may be deterministic or stochastic, depend on  $h$  and  $u$ , may be chosen per tool, and report any exposed domain information, including facts unrelated to the call's direct effects. In our experiments, one function serves every call on  $\mathcal{T}$  and returns the open identities and their frontier versions.

**Task Activity Graph (TAG).** The TAG  $\mathcal{T}$  contains task versions  $q$ , task actions  $a$ , and evidence attachments  $\phi$ :

$$q = (k, \delta, \sigma) \in \mathcal{Q}, \quad a = (h, u) \in \mathcal{A}, \quad u \in \mathcal{U}_T, \\ \delta = (g, c), \quad c(g, \sigma) \in \Omega, \quad \phi = (s, x, q) \in \Phi.$$

Here  $k$  is the persistent task identifier,  $g$  is the goal,  $\sigma$  is the reported state, and  $c$  is an evaluation rule whose result may be a score or statement. These fields may be expressed through language, predicates, programs, or other task-specific objects. Each  $q$  has a version identifier separate from its displayed fields. Versions are immutable, and distinct versions may have identical fields.

Each task-call occurrence creates exactly one action. For  $a \in \mathcal{A}$ , let  $Q_a^{\text{in}}$  be its input versions and  $Q_a^{\text{out}}$  its created versions. Inputs must exist at call issuance, and outputs have fresh version identifiers. Initial task versions may be supplied externally and presented through initial events; every noninitial version has exactly one producing action. The task incidences are

$$\mathcal{L}_T = \bigcup_{a \in \mathcal{A}} (\{q \rightarrow a : q \in Q_a^{\text{in}}\} \cup \{a \rightarrow q : q \in Q_a^{\text{out}}\}).$$

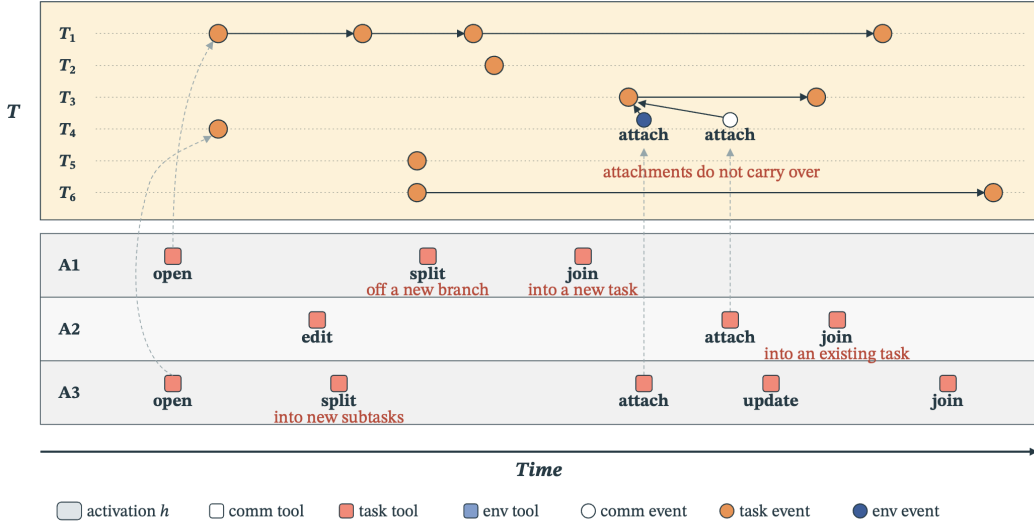


Figure 11: Task evolution and agent attribution. Task versions are arranged by task identity above agent-issued calls. Selected task and attribution links are shown; task-action nodes are suppressed.

For  $Q = Q_a^{\text{out}}$ , the update  $\mathcal{T} \oplus (a, Q)$  adds  $a$ ,  $Q$ , and their input–output incidences. The created set  $Q$  is separate from the returned event set  $E_u$ ; either may be empty independently of the other. Versions are not events:  $Q$  and  $\mathcal{E}$  are disjoint, and  $\mathcal{D}$  and  $\mathcal{T}$  may represent the same information in different ways. Returned events may describe task versions, but need not enumerate  $Q$ .

**Task references.** Version transformations and attachments address exact versions, whereas CLOSE addresses a task identity. Attachment targets and identities targeted by CLOSE must exist at call issuance. Applications may impose additional access restrictions.

Events may reference task identities or exact versions. References must resolve at event creation and may name identities or versions introduced by the same call. A reference alone does not present the target’s fields to the activation.

**Version transformations.** Version transformations may use historical versions outside the current frontier and versions of closed tasks. Consuming a version records its use as an input; it does not delete the version or prevent later reuse.

Using a historical version does not restore  $\mathcal{Z}$ . Restoration requires environment actions and may be impossible after irreversible effects.

The arrows in Table 5 suppress action nodes.

OPEN assigns a fresh task identity. EDIT $_{\delta}$  creates  $q' = (k, \delta', \sigma)$ , and UPDATE $_{\sigma}$  creates  $q' = (k, \delta, \sigma')$ . SPLIT creates  $m \geq 2$  versions with pairwise distinct task identities: at most one continues the input identity, and all others are fresh. JOIN takes  $m \geq 2$  input versions with pairwise distinct task identities and creates one version under an input identity or a fresh identity. An identity is continued only from its frontier version, so it has at most one frontier version at any time. A transformation of a consumed version branches: EDIT $_{\delta}$  and UPDATE $_{\sigma}$  then create  $q'$  under a fresh identity, SPLIT gives every part a fresh identity, and JOIN may take the consumed version as an input but not continue its identity. Concurrent transformations of one version thus yield distinct tasks. Goal satisfaction and the semantic adequacy of a split or join require separate assessment.

Table 5: Version-transforming task tools.

| Task tool                                           | Version transition                   |
|-----------------------------------------------------|--------------------------------------|
| OPEN( $\delta, \sigma$ )                            | $\emptyset \rightarrow q$            |
| EDIT $_{\delta}(q, \delta')$                        | $q \rightarrow q'$                   |
| UPDATE $_{\sigma}(q, \sigma')$                      | $q \rightarrow q'$                   |
| SPLIT( $q, \{(k_j, \delta_j, \sigma_j)\}_{j=1}^m$ ) | $q \rightarrow \{q_1, \dots, q_m\}$  |
| JOIN( $\{q_j\}_{j=1}^m, k', \delta', \sigma'$ )     | $\{q_1, \dots, q_m\} \rightarrow q'$ |

**Task lifecycle.** Each task identity  $k$  is initialized as open. The tool  $\text{CLOSE}(k) \in \mathcal{U}_T$  marks it closed, declaring that further work is no longer required across all versions of  $k$ . Closure does not certify goal completion. Version transformations preserve existing identities' status. A close call has  $Q_a^{\text{in}} = Q_a^{\text{out}} = \emptyset$ , with its target  $k$  recorded in  $u$ .

**Open tasks and version frontier.** Let  $\mathcal{K}_t$  be the set of task identities that are open at time  $t$ ; all other introduced identities are closed. The structural version frontier is

$$\mathcal{F}_t = \mathcal{Q}_t \setminus \bigcup_{a \in \mathcal{A}_t} Q_a^{\text{in}}.$$

The frontier for open tasks is  $\{q = (k, \delta, \sigma) \in \mathcal{F}_t : k \in \mathcal{K}_t\}$ . Closing leaves  $\mathcal{F}_t$  unchanged. An open identity need not have a frontier version.

**Evidence attachments.** An attachment  $\phi = (s, x, q)$  records a declared source  $s$ , arbitrary payload  $x$ , and exact target  $q \in \mathcal{Q}$ . The source may identify an agent, activation, environment call, or other implementation-defined provenance; it need not equal the attaching agent. The payload need not be a DIG event. Multiple attachments may target the same version.

The call  $u = \text{attach}_h(\phi)$  adds  $\phi$  to  $\Phi$ , written  $\mathcal{T} \oplus \phi$ , and is recorded in  $U_h$  with its returned events  $E_u$ , obtained as for any call on  $\mathcal{T}$ . Reattaching the same triple adds a call without duplicating the attachment. Evidence at a successor  $q'$  requires a separate attachment  $(s, x, q')$ ; its validity is assessed under the reuse conditions in Section 4.3.

**Information tools.** The class  $\mathcal{U}_\phi$  consists of  $\text{send}_h$ ,  $\text{attach}_h$ , and  $\text{observe}_h$ , bound to the issuing activation  $h$ . A call  $\text{observe}_h$  reads  $\mathcal{T}$  without changing it; it creates no action and is recorded in  $U_h$  with its returned events  $E_u$ . A call  $\text{send}_h(e; J)$  forwards an available event from any domain to intended recipients  $J \subseteq \mathcal{I}$ . It records the recipients and an empty return. Delivery, which may be delayed, includes  $e$  in the input set of a new receiving activation  $h'_j$  of  $j \in J$ , without producing another event or recording another call. Implementations may update auxiliary delivery metadata while retaining earlier call and delivery records.

Input edges may also record initial presentation or runtime-defined routing; they do not by themselves identify a SEND occurrence. Attributing a runtime routing decision requires recording it under an identified activation.

Sending a new or revised message requires an event-producing call followed by SEND; automatic forwarding records both calls. Appendix ?? uses a PUBLISH environment call to create such events.

**Shared environment.** The component  $\mathcal{Z}$  is the shared environment state. For a call  $u \in \mathcal{U}_Z$ , the transition to  $\mathcal{Z}'$ , with returned events  $E_u$ , is written

$$\mathcal{Z} \xrightarrow{(h,u)/E_u} \mathcal{Z}'.$$

State changes may be deterministic or stochastic; a query may leave  $\mathcal{Z}$  unchanged. Each transition describes an effect at the chosen recording granularity; separately visible intermediate effects require separate steps. The environment may also change without an agent call. Such changes enter DIG only through subsequent observations.

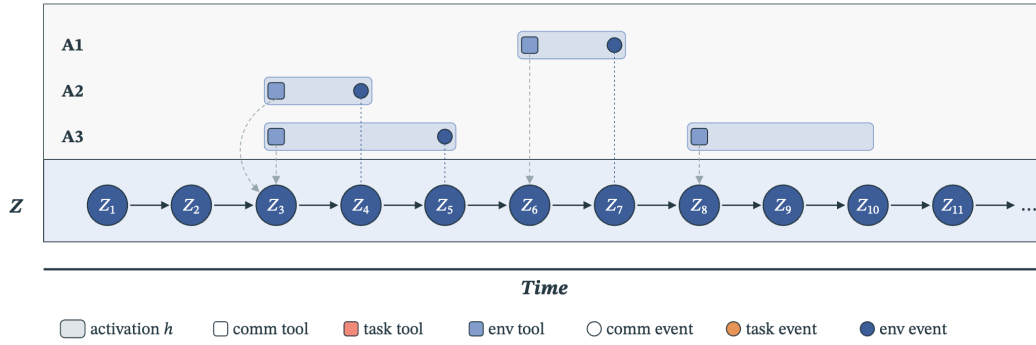


Figure 12: Environment interaction. Agent-issued calls may overlap, and feedback may be partial or empty. The displayed state sequence includes evolution that need not be externalized in DIG.